AFRILANG Stdlib

std/c/simmonte

Bibliothèque AFRILANG `std/c/simmonte` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : estimatePi, inte

`import "std/c/simmonte"` · `use simmonte`

- `estimatePi(samples number, seed number) ? number`
- `integrate(samples number, seed number, a number, b number) ? number`
- `randomNormal(n number, seed number) ? list number`
- `confidence95(samples list number) ? list number`
- `bootstrapMean(data list number, samples number, seed number) ? number`
- `varianceReduction(samples number, seed number) ? number`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/simmonte` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : estimatePi, inte

`import "std/c/simmonte"` · `use simmonte`

- `estimatePi(samples number, seed number) ? number`
- `integrate(samples number, seed number, a number, b number) ? number`
- `randomNormal(n number, seed number) ? list number`
- `confidence95(samples list number) ? list number`
- `bootstrapMean(data list number, samples number, seed number) ? number`
- `varianceReduction(samples number, seed number) ? number`

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/simmonte"
use simmonte
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? estimatePi(samples number, seed number) ? number
? integrate(samples number, seed number, a number, b number) ? number
? randomNormal(n number, seed number) ? list
? confidence95(samples list number) ? list
? bootstrapMean(data list number, samples number, seed number) ? number
? varianceReduction(samples number, seed number) ? number
```

4. Exemples d'utilisation

```
import "std/c/simmonte"
use simmonte

create result = estimatePi(/* args */)
say result

create result = integrate(/* args */)
say result

create result = randomNormal(/* args */)
say result

create result = confidence95(/* args */)
say result

create result = bootstrapMean(/* args */)
say result

create result = varianceReduction(/* args */)
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/simmonte"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module simmonte

function estimatePi(samples number, seed number) returns number
end
```

```
function integrate(samples number, seed number, a number, b number) returns number
end

function randomNormal(n number, seed number) returns list number
end

function confidence95(samples list number) returns list number
end

function bootstrapMean(data list number, samples number, seed number) returns number
end

function varianceReduction(samples number, seed number) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/simmonte` (tier complex, category complex). Exports 6 function(s): estimatePi, integrate, random

`import "std/c/simmonte"` · `use simmonte`

- `estimatePi(samples number, seed number) ? number`
- `integrate(samples number, seed number, a number, b number) ? number`
- `randomNormal(n number, seed number) ? list number`
- `confidence95(samples list number) ? list number`
- `bootstrapMean(data list number, samples number, seed number) ? number`
- `varianceReduction(samples number, seed number) ? number`

2. Installation & import

Add the module to your program:

```
import "std/c/simmonte"
use simmonte
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? estimatePi(samples number, seed number) ? number
? integrate(samples number, seed number, a number, b number) ? number
? randomNormal(n number, seed number) ? list
? confidence95(samples list number) ? list
? bootstrapMean(data list number, samples number, seed number) ? number
? varianceReduction(samples number, seed number) ? number
```

4. Usage examples

```
import "std/c/simmonte"
use simmonte

create result = estimatePi(/* args */)
say result

create result = integrate(/* args */)
say result

create result = randomNormal(/* args */)
say result

create result = confidence95(/* args */)
say result

create result = bootstrapMean(/* args */)
say result

create result = varianceReduction(/* args */)
say result
```

5. Best practices

```
? Prefer `import "std/c/simmonte"` at the top of your file.
? Run `afrilang check` before shipping.
? Combine with related stdlib modules from the same category.
? See /stdlib/ for the live source and playground demos.
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module simmonte

function estimatePi(samples number, seed number) returns number
end
```

```
function integrate(samples number, seed number, a number, b number) returns number
end

function randomNormal(n number, seed number) returns list number
end

function confidence95(samples list number) returns list number
end

function bootstrapMean(data list number, samples number, seed number) returns number
end

function varianceReduction(samples number, seed number) returns number
end
end
```